K.R. MANGALAM UNIVERSITY

School of Engineering and Technology

Lab Assignment – 3:
Graph Algorithms

Submitted by:
Satyam Singh
(2401420022)

Submitted to:
Dr. Aarti

Assignment Tasks

Task 1: Graph Representation

Overview

This task demonstrates two standard ways to represent a directed graph in memory. A 6-node directed graph (A–F) is built and displayed using both an Adjacency List and an Adjacency Matrix, highlighting the trade-offs between the two approaches.

Graph Structure

Nodes: A, B, C, D, E, F

Directed Edges: A→B, A→C, B→D, B→E, C→F, E→F

Adjacency List — Implementation

Each node maps to a Python list of its direct neighbours. Space: O(V + E).

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}
for node in graph:
    print(node, '->', graph[node])
```

Adjacency Matrix — Implementation

A 6×6 integer matrix where matrix[i][j] = 1 if edge i→j exists, else 0. Space: $O(V^2)$.

```
nodes  = ['A','B','C','D','E','F']
matrix = [
    [0,1,1,0,0,0],  # A
    [0,0,0,1,1,0],  # B
    [0,0,0,0,0,1],  # C
    [0,0,0,0,0,0],  # D
    [0,0,0,0,0,1],  # E
    [0,0,0,0,0,0]   # F
]
```

Code Screenshot

```python
# -------- TASK 1 --------
# Adjacency List Representation

graph = {
    'A':['B','C'],
    'B':['D','E'],
    'C':['F'],
    'D':[],
    'E':['F'],
    'F':[]
}

print("Adjacency List:")
for node in graph:
    print(node, "->", graph[node])


# Adjacency Matrix Representation

nodes=['A','B','C','D','E','F']

matrix=[
[0,1,1,0,0,0],
[0,0,0,1,1,0],
[0,0,0,0,0,1],
[0,0,0,0,0,0],
[0,0,0,0,0,1],
[0,0,0,0,0,0]
]

print("Adjacency Matrix:")
for row in matrix:
    print(row)
```

Comparison

Property	Adjacency List	Adjacency Matrix
Space	O(V + E)	$O(V^2)$
Edge Lookup	O(degree(u))	O(1)
Add/Remove Edge	O(1) / O(E)	O(1)
Iterate Neighbours	O(degree(u))	O(V)

Best For	Sparse graphs	Dense graphs

Observation

The adjacency list is memory-efficient for sparse graphs (few edges relative to V^2) and is preferred when iterating over neighbours, as in BFS/DFS. The adjacency matrix allows O(1) edge existence queries but wastes memory for sparse graphs. For this 6-node graph with only 6 edges, the list uses far less memory than the 36-cell matrix.

Result / Output

```
PS C:\Users\Satyam\Documents\ada> python -u "c:\Users\Satyam\Documents\ada\assignment_3.py"
Adjacency List:
A -> ['B', 'C']
B -> ['D', 'E']
C -> ['F']
D -> []
E -> ['F']
F -> []
Adjacency Matrix:
[0, 1, 1, 0, 0, 0]
[0, 0, 0, 1, 1, 0]
[0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 0]
```

Task 2: Graph Traversal Algorithms — BFS and DFS

2A — Breadth-First Search (BFS)

BFS explores a graph level-by-level from the source node, using a FIFO queue. It guarantees that every node is visited at its minimum edge-distance from the source.

Algorithm:

1. Enqueue the start node; mark it visited.
2. Dequeue a node; print it; enqueue all unvisited neighbours.
3. Repeat until the queue is empty.

Complexity:

Time: O(V + E) | Space: O(V)

BFS Code

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue   = deque([start])
    while queue:
        node = queue.popleft()
        if node not in visited:
            print(node, end=' ')
            visited.add(node)
            for neighbour in graph[node]:
                queue.append(neighbour)
```

```python
def bfs(graph,start):

    visited=set()
    queue=deque([start])

    while queue:

        node=queue.popleft()

        if node not in visited:
            print(node,end=" ")
            visited.add(node)

            for neighbour in graph[node]:
                queue.append(neighbour)

print("BFS Traversal:")
bfs(graph,'A')
```

2B — Depth-First Search (DFS)

DFS explores as deep as possible along each branch before backtracking. The recursive implementation implicitly uses the call stack.

Algorithm:

1. Mark the node visited; print it.
2. Recursively visit each unvisited neighbour.
3. Backtrack when all neighbours are visited.

Complexity:

Time: O(V + E) | Space: O(V) recursion depth

DFS Code

```python
def dfs(graph, node, visited=set()):
    if node not in visited:
        print(node, end=' ')
        visited.add(node)
        for neighbour in graph[node]:
            dfs(graph, neighbour, visited)
```

```python
def dfs(graph,node,visited=set()):

    if node not in visited:
        print(node,end=" ")
        visited.add(node)

        for neighbour in graph[node]:
            dfs(graph,neighbour,visited)

print("DFS Traversal:")
dfs(graph,'A')
```

BFS vs DFS Comparison

Property	BFS	DFS
Data Structure	Queue (FIFO)	Stack / Recursion
Traversal Style	Level-by-level	Deepest branch first
Shortest Path?	Yes (unweighted graphs)	Not guaranteed
Memory (worst)	O(V) — wide frontier	O(V) — deep call stack
Best Use Case	Shortest path, level order	Cycle detection, topological sort

Observation

BFS visits A → B → C → D → E → F (level order). DFS visits A → B → D → E → F → C (deep first). Both reach the same set of nodes but in different orders. BFS is preferable when the shortest route matters; DFS is preferable for exhaustive path searches and dependency resolution.

Result

```
BFS Traversal:
A B C D E F
DFS Traversal:
A B D E F C
```

Task 3: Topological Sorting

Overview

Topological sorting produces a linear ordering of nodes in a Directed Acyclic Graph (DAG) such that for every directed edge u → v, node u appears before v. It is implemented via DFS with post-order stack insertion.

Real-World Use

Task scheduling, build-system dependency resolution (e.g., Makefile), course prerequisite ordering, and package manager dependency graphs all rely on topological sorting.

Algorithm

1. For each unvisited node, run DFS.
2. After all neighbours of a node are visited, push the node onto a stack.
3. Reverse the stack — this gives the topological order.

Complexity:

Time: O(V + E) | Space: O(V)

Graph Used (graph2)

Nodes: A, B, C, D, E, F, G, H

Edges: A→C, B→C, B→D, C→E, D→F, E→H, E→F, F→G

Code

```
def topological_sort(graph):
    visited = set()
    stack   = []
    def dfs(node):
        visited.add(node)
        for neighbour in graph[node]:
            if neighbour not in visited:
                dfs(neighbour)
        stack.append(node)          # post-order push
    for node in graph:
        if node not in visited:
            dfs(node)
    return stack[::-1]              # reverse = topological order
```

Observation

A valid topological order for graph2 is: B → A → D → C → E → F → G → H (exact order can vary based on dictionary iteration order, but all dependencies are satisfied). Every node appears before the nodes it points to, confirming correct implementation.

Result / Output

```
Topological Order: ['B', 'D', 'A', 'C', 'E', 'F', 'G', 'H']
```

Task 4: Shortest Path Algorithm — Dijkstra's

Overview

Dijkstra's algorithm computes the shortest path from a single source to all other nodes in a weighted graph with non-negative edge weights. It uses a min-heap (priority queue) to always extend the currently shortest known path.

Real-World Use

GPS navigation (Google Maps, Waze), network routing protocols (OSPF), and emergency vehicle path planning all use Dijkstra's or its variants.

Algorithm

1. Set distance to source = 0; all others = ∞.
2. Push (0, source) into a min-heap.
3. Pop minimum-distance node. For each neighbour, compute candidate distance.
4. If candidate < known distance, update and push to heap.
5. Repeat until heap is empty.

Complexity:

Time: O((V + E) log V) using binary min-heap | Space: O(V)

Graph Used (graph3)

Nodes: A, B, C, D, E, F

Directed Weighted Edges: A→B(4), A→C(2), B→C(5), B→D(10), C→E(3), E→D(4), D→F(11)

Code

```
import heapq

def dijkstra(graph, start):
    pq        = [(0, start)]
    distances = {node: float('inf') for node in graph}
    distances[start] = 0
    while pq:
        current_dist, current_node = heapq.heappop(pq)
        for neighbour, weight in graph[current_node].items():
            dist = current_dist + weight
            if dist < distances[neighbour]:
                distances[neighbour] = dist
                heapq.heappush(pq, (dist, neighbour))
```

```
    return distances
```

Expected Shortest Distances from A

Destination Node	Shortest Distance	Path
A	0	A (source)
B	4	A → B
C	2	A → C
D	9	A → C → E → D
E	5	A → C → E
F	20	A → C → E → D → F

Time Complexity Behaviour

For small graphs (V < 100), Dijkstra runs in microseconds. For large sparse graphs (V ~ 10^6), the O((V+E) log V) complexity is acceptable. Dijkstra cannot handle negative edge weights — Bellman-Ford must be used in that case.

Observation

The path to D via C→E→D (cost 9) is cheaper than the direct path A→B→D (cost 14), demonstrating that Dijkstra correctly explores all routes to find the true minimum. The priority queue ensures the optimal path is always extended first.

Result:

```
Shortest Paths: {'A': 0, 'B': 4, 'C': 2, 'D': 9, 'E': 5, 'F': 20}
```

Task 5: Minimum Spanning Tree — Prim's Algorithm

Overview

Prim's algorithm builds a Minimum Spanning Tree (MST) for a connected, undirected, weighted graph. It greedily grows the MST one edge at a time by always picking the cheapest edge that connects an already-visited node to an unvisited node.

Real-World Use

Network cable layout (minimising wire cost), road construction linking cities at minimum cost, and cluster analysis in machine learning all use MST algorithms.

Algorithm

1. Start at an arbitrary node; add its edges to a min-heap.
2. Pop the minimum-weight edge (frm → to).
3. If 'to' is unvisited: add edge to MST, mark visited, push to's edges to heap.
4. Repeat until all V nodes are visited (MST has V−1 edges).

Complexity:

Time: O((V + E) log V) | Space: O(V + E)

Graph Used (graph4)

Nodes: A, B, C, D (Undirected, Weighted)

Edges: A-B(2), A-C(3), B-C(1), B-D(1), C-D(4)

Code

```
def prim(graph, start):
    visited  = set([start])
    edges    = []
    min_heap = []
    total    = 0
    for to, weight in graph[start].items():
        heapq.heappush(min_heap, (weight, start, to))
    while min_heap:
        weight, frm, to = heapq.heappop(min_heap)
        if to not in visited:
            visited.add(to)
            edges.append((frm, to, weight))
            total += weight
            for next_node, next_weight in graph[to].items():
                if next_node not in visited:
```

```
                    heapq.heappush(min_heap, (next_weight, to, next_node))
    return edges, total
```

Expected MST Output

Edge	Weight	Included in MST?	Reason
A — B	2	Yes	Cheapest edge from A
B — C	1	Yes	Cheapest edge connecting C
B — D	1	Yes	Cheapest edge connecting D
A — C	3	No	Would create a cycle
C — D	4	No	Would create a cycle

Total MST Cost: 2 + 1 + 1 = 4

Prim vs Kruskal

Property	Prim's	Kruskal's
Approach	Grow tree from source	Sort all edges, add greedily
Data Structure	Min-heap + visited set	Sorted edge list + Union-Find
Complexity	O((V+E) log V)	O(E log E)
Better For	Dense graphs	Sparse graphs
Implementation	Simpler with adjacency list	Simpler with edge list

Observation

Prim's correctly selects edges A-B(2), B-C(1), B-D(1) for a total cost of 4, which is the minimum possible to connect all 4 nodes. The algorithm always terminates with exactly V−1 = 3 edges. Kruskal's would produce the same MST but via a different internal process (sorting edges globally rather than growing from a node).

Result

```
MST Edges: [('A', 'B', 2), ('B', 'C', 1), ('B', 'D', 1)]
Total Cost: 4
PS C:\Users\Satyam\Documents\ada>
```

GitHub Repository: https://github.com/Code-trac/ada_lab